L10: Daten über Tabellengrenzen hinweg kombinieren – Von Subqueries zu CTEs zu Joins

Session 10 – Lecture Dauer: 90 Minuten **Lernziele:** LZ 2 (Relationale DB & SQL praktisch anwenden)
Block: 2 – SQL Einführung & Grundlagen

Willkommen zur zehnten Session! Bisher haben Sie mit einzelnen Tabellen gearbeitet – SELECT, WHERE, GROUP BY, alles auf einer Tabelle. Das war wichtig zum Lernen, aber die wahre Macht relationaler Datenbanken liegt woanders: in Beziehungen.

Kunden haben Bestellungen. Bestellungen haben Positionen. Produkte gehören zu Kategorien. All diese Informationen leben in verschiedenen Tabellen – aber wie kombinieren wir sie? Es gibt drei Hauptansätze: Subqueries, CTEs und Joins.

In dieser Session lernen Sie alle drei Techniken kennen – und verstehen, wann Sie welche einsetzen. Wir beginnen mit dem intuitiven Weg (Subqueries), zeigen dessen Grenzen, verbessern ihn mit CTEs, und landen schließlich bei der elegantesten Lösung: Joins.

Los geht's mit der Frage: Warum müssen wir Daten überhaupt kombinieren?

Warum Daten kombinieren?

Stellen Sie sich vor, Sie speichern alles in einer Tabelle: Kundendaten, Bestellungen, Produktdetails – alles zusammen. Was passiert?

Problem 1: Redundanz Sie speichern die Kundenadresse bei jeder Bestellung neu. Zieht der Kunde um, müssen Sie Dutzende Zeilen aktualisieren.

Problem 2: Inkonsistenz Bei manchen Bestellungen steht „Berlin„, bei anderen „Berln“ – Tippfehler.

Problem 3: Update-Anomalien Sie ändern den Preis eines Produkts – aber welche Bestellungen bekommen den neuen Preis? Die alten sollten den alten Preis behalten!

Die Lösung? Normalisierung. Wir teilen Daten in mehrere Tabellen auf. Jede Tabelle hat eine klar definierte Verantwortung. Beziehungen werden über Foreign Keys hergestellt.

Normalisierung führt zu mehreren Tabellen.

Joins rekonstruieren die Informationen.

Unser E-Commerce-Schema

Für alle Beispiele heute nutzen wir ein realistisches E-Commerce-Schema mit sieben normalisierten Tabellen inklusive einer N:M-Beziehung über eine Junction Table.

```sql
-- Locations: Normalisierte Orte mit PLZ
CREATE TABLE locations (
  location_id INTEGER PRIMARY KEY,
  city TEXT NOT NULL,
  postal_code TEXT NOT NULL,
  country TEXT DEFAULT 'Germany'
);

-- Categories: Normalisierte Produktkategorien
CREATE TABLE categories (
  category_id INTEGER PRIMARY KEY,
  category_name TEXT NOT NULL UNIQUE,
  description TEXT
);

-- Customers: Erweitert mit strukturierten Adressdaten
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  email TEXT UNIQUE,
  street TEXT,
  street_number TEXT,
  location_id INTEGER,
  FOREIGN KEY (location_id) REFERENCES locations(location_id)
);

-- Orders: Unverändert
CREATE TABLE orders (
  order_id INTEGER PRIMARY KEY,
  customer_id INTEGER,
  order_date DATE,
  total_amount DECIMAL(10,2),
  status TEXT,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);

-- Products: Ohne direkte Category-Referenz (N:M über Junction Table
CREATE TABLE products (
  product_id INTEGER PRIMARY KEY,
  product_name TEXT NOT NULL,
  price DECIMAL(10,2)
);
```

```sql
-- Product Categories: Junction Table für N:M-Beziehung
CREATE TABLE product_categories (
  product_id INTEGER,
  category_id INTEGER,
  PRIMARY KEY (product_id, category_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id),
  FOREIGN KEY (category_id) REFERENCES categories(category_id)
);

-- Order Items: Unverändert
CREATE TABLE order_items (
  order_item_id INTEGER PRIMARY KEY,
  order_id INTEGER,
  product_id INTEGER,
  quantity INTEGER,
  line_total DECIMAL(10,2),
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
);

-- Sample Data: Locations
INSERT INTO locations(location_id, city, postal_code, country) VALUE
  (1, 'Berlin', '10115', 'Germany'),
  (2, 'Hamburg', '20095', 'Germany'),
  (3, 'Munich', '80331', 'Germany'),
  (4, 'Cologne', '50667', 'Germany'),
  (5, 'Frankfurt', '60311', 'Germany');

-- Sample Data: Categories
INSERT INTO categories(category_id, category_name, description) VALU
  (1, 'Electronics', 'Electronic devices and accessories'),
  (2, 'Furniture', 'Office and home furniture'),
  (3, 'Stationery', 'Office supplies and paper products'),
  (4, 'Office Equipment', 'Professional office tools and devices');

-- Sample Data: Customers (mit strukturierten Adressen)
INSERT INTO customers(customer_id, first_name, last_name, email, str
  street_number, location_id) VALUES
  (1, 'Alice', 'Anderson', 'alice@email.com', 'Unter den Linden', '4
    ),
  (2, 'Bob',   'Brown',    'bob@email.com',   'Reeperbahn',       '1
    ),
  (3, 'Carol', 'Clark',    'carol@email.com', 'Marienplatz',       '
    ),
  (4, 'David', 'Davis',    'david@email.com', 'Hohe Straße',     '12
    ),
  (5, 'Emma',  'Evans',    'emma@email.com',  'Zeil',             '9
    );

-- Sample Data: Orders (Note: Customer 5 has NO orders!)
```

```sql
INSERT INTO orders(order_id, customer_id, order_date, total_amount, ) VALUES
  (101, 1, '2024-01-15', 299.99, 'completed'),
  (102, 1, '2024-02-20', 149.50, 'completed'),
  (103, 2, '2024-01-22', 499.99, 'completed'),
  (104, 3, '2024-03-10',  89.99, 'pending'  ),
  (105, 4, '2024-03-15', 199.99, 'completed');

-- Sample Data: Products (ohne direkte Category-Referenz)
INSERT INTO products(product_id, product_name, price) VALUES
  (1, 'Laptop',     999.99),
  (2, 'Mouse',       29.99),
  (3, 'Keyboard',    79.99),
  (4, 'Monitor',    299.99),
  (5, 'Desk Chair', 199.99),
  (6, 'Notebook',     9.99);

-- Sample Data: Product Categories (N:M-Beziehungen)
INSERT INTO product_categories(product_id, category_id) VALUES
  (1, 1),  -- Laptop → Electronics
  (1, 4),  -- Laptop → Office Equipment
  (2, 1),  -- Mouse → Electronics
  (2, 4),  -- Mouse → Office Equipment
  (3, 1),  -- Keyboard → Electronics
  (3, 4),  -- Keyboard → Office Equipment
  (4, 1),  -- Monitor → Electronics
  (4, 4),  -- Monitor → Office Equipment
  (5, 2),  -- Desk Chair → Furniture
  (5, 4),  -- Desk Chair → Office Equipment
  (6, 3);  -- Notebook → Stationery (nur eine Kategorie!)

-- Sample Data: Order Items
INSERT INTO order_items(order_item_id, order_id, product_id, quantit line_total) VALUES
  (1, 101, 4, 1, 299.99),
  (2, 102, 2, 2,  59.98),
  (3, 102, 3, 1,  79.99),
  (4, 103, 1, 1, 999.99),
  (5, 104, 6, 5,  49.95),
  (6, 105, 5, 1, 199.99);
```

```sql
-- Locations: Normalisierte Orte mit PLZ
CREATE TABLE locations (
  location_id INTEGER PRIMARY KEY,
  city TEXT NOT NULL,
  postal_code TEXT NOT NULL,
  country TEXT DEFAULT 'Germany'
)
```

ok

```sql
-- Categories: Normalisierte Produktkategorien
CREATE TABLE categories (
  category_id INTEGER PRIMARY KEY,
  category_name TEXT NOT NULL UNIQUE,
  description TEXT
)
```

ok

```sql
-- Customers: Erweitert mit strukturierten Adressdaten
CREATE TABLE customers (
  customer_id INTEGER PRIMARY KEY,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  email TEXT UNIQUE,
  street TEXT,
  street_number TEXT,
  location_id INTEGER,
  FOREIGN KEY (location_id) REFERENCES locations(location_id)
)
```

ok

```sql
-- Orders: Unverändert
CREATE TABLE orders (
  order_id INTEGER PRIMARY KEY,
  customer_id INTEGER,
  order_date DATE,
  total_amount DECIMAL(10,2),
  status TEXT,
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
)
```

ok

```sql
-- Products: Ohne direkte Category-Referenz (N:M über Junction Table)
CREATE TABLE products (
  product_id INTEGER PRIMARY KEY,
  product_name TEXT NOT NULL,
  price DECIMAL(10,2)
)
```

ok

```
-- Product Categories: Junction Table für N:M-Beziehung
CREATE TABLE product_categories (
  product_id INTEGER,
  category_id INTEGER,
  PRIMARY KEY (product_id, category_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id),
  FOREIGN KEY (category_id) REFERENCES categories(category_id)
)
```

ok

```
-- Order Items: Unverändert
CREATE TABLE order_items (
  order_item_id INTEGER PRIMARY KEY,
  order_id INTEGER,
  product_id INTEGER,
  quantity INTEGER,
  line_total DECIMAL(10,2),
  FOREIGN KEY (order_id) REFERENCES orders(order_id),
  FOREIGN KEY (product_id) REFERENCES products(product_id)
)
```

ok

```
-- Sample Data: Locations
INSERT INTO locations(location_id, city, postal_code, country) VALUES
  (1, 'Berlin', '10115', 'Germany'),
  (2, 'Hamburg', '20095', 'Germany'),
  (3, 'Munich', '80331', 'Germany'),
  (4, 'Cologne', '50667', 'Germany'),
  (5, 'Frankfurt', '60311', 'Germany')
```

ok

```
-- Sample Data: Categories
INSERT INTO categories(category_id, category_name, description) VALUES
  (1, 'Electronics', 'Electronic devices and accessories'),
  (2, 'Furniture', 'Office and home furniture'),
  (3, 'Stationery', 'Office supplies and paper products'),
  (4, 'Office Equipment', 'Professional office tools and devices')
```

ok

```
-- Sample Data: Customers (mit strukturierten Adressen)
INSERT INTO customers(customer_id, first_name, last_name, email,
street, street_number, location_id) VALUES
  (1, 'Alice', 'Anderson', 'alice@email.com', 'Unter den Linden', '42',
1),
  (2, 'Bob',   'Brown',    'bob@email.com',   'Reeperbahn',       '15',
```

```
2),
  (3, 'Carol', 'Clark',    'carol@email.com', 'Marienplatz',       '8',
3),
  (4, 'David', 'Davis',    'david@email.com', 'Hohe Straße',     '123',
4),
  (5, 'Emma',  'Evans',    'emma@email.com',  'Zeil',             '99',
5)
```

ok

```
-- Sample Data: Orders (Note: Customer 5 has NO orders!)
INSERT INTO orders(order_id, customer_id, order_date, total_amount,
status) VALUES
  (101, 1, '2024-01-15', 299.99, 'completed'),
  (102, 1, '2024-02-20', 149.50, 'completed'),
  (103, 2, '2024-01-22', 499.99, 'completed'),
  (104, 3, '2024-03-10',  89.99, 'pending'  ),
  (105, 4, '2024-03-15', 199.99, 'completed')
```

ok

```
-- Sample Data: Products (ohne direkte Category-Referenz)
INSERT INTO products(product_id, product_name, price) VALUES
  (1, 'Laptop',     999.99),
  (2, 'Mouse',       29.99),
  (3, 'Keyboard',    79.99),
  (4, 'Monitor',    299.99),
  (5, 'Desk Chair', 199.99),
  (6, 'Notebook',     9.99)
```

ok

```
-- Sample Data: Product Categories (N:M-Beziehungen)
INSERT INTO product_categories(product_id, category_id) VALUES
  (1, 1),  -- Laptop → Electronics
  (1, 4),  -- Laptop → Office Equipment
  (2, 1),  -- Mouse → Electronics
  (2, 4),  -- Mouse → Office Equipment
  (3, 1),  -- Keyboard → Electronics
  (3, 4),  -- Keyboard → Office Equipment
  (4, 1),  -- Monitor → Electronics
  (4, 4),  -- Monitor → Office Equipment
  (5, 2),  -- Desk Chair → Furniture
  (5, 4),  -- Desk Chair → Office Equipment
  (6, 3)
```

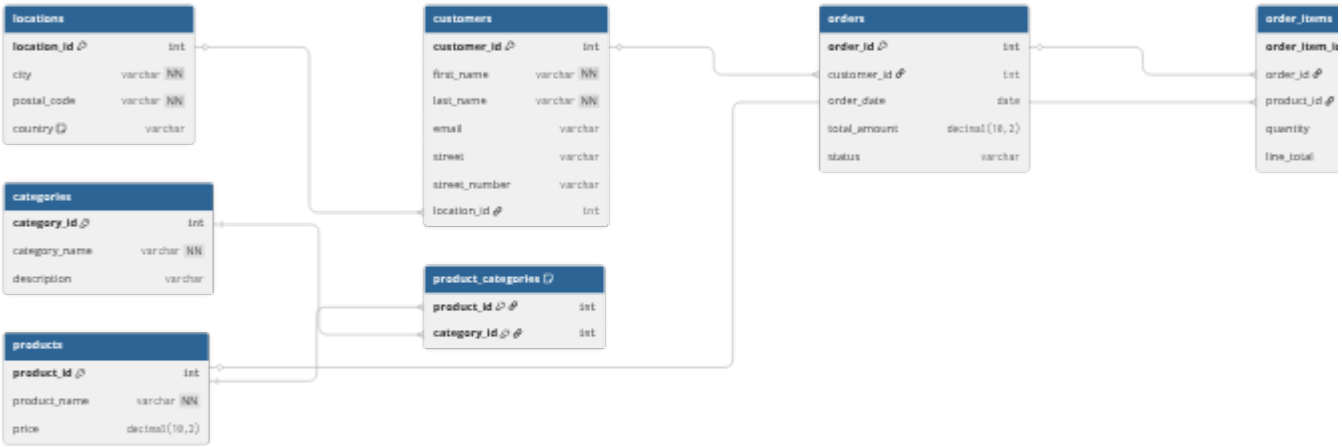
ok

```
-- Notebook → Stationery (nur eine Kategorie!)

-- Sample Data: Order Items
INSERT INTO order_items(order_item_id, order_id, product_id, quantity,
```

```
line_total) VALUES
  (1, 101, 4, 1, 299.99),
  (2, 102, 2, 2,  59.98),
  (3, 102, 3, 1,  79.99),
  (4, 103, 1, 1, 999.99),
  (5, 104, 6, 5,  49.95),
  (6, 105, 5, 1, 199.99)
ok
```

Beziehungen:



dbdiagram.io

dbdiagram.io

- Ein Ort kann viele Kunden haben (1:N)
- **Ein Produkt kann viele Kategorien haben (N:M über product_categories)**
- **Eine Kategorie kann viele Produkte haben (N:M über product_categories)**
- Ein Kunde kann viele Bestellungen haben (1:N)
- Eine Bestellung hat viele Positionen (1:N)
- Ein Produkt kann in vielen Positionen vorkommen (N:M über order_items)

Diese Struktur ist typisch für relationale Datenbanken. Aber wie kombinieren wir diese Informationen? Schauen wir uns vier Ansätze an – beginnend mit dem ältesten, aber wichtigsten zum Verstehen.

Technik 0: Verknüpfen von Tabellen über `FROM`

Bevor wir zu modernen Techniken kommen, müssen wir die Basis verstehen: Wie kombiniert SQL überhaupt Tabellen? Die Antwort liegt im FROM. Sie können mehrere Tabellen einfach durch Kommata trennen – und erhalten das kartesische Produkt.

Das kartesische Produkt: Alle Kombinationen

Wenn Sie zwei Tabellen im FROM auflisten, verbindet SQL jede Zeile der ersten Tabelle mit jeder Zeile der zweiten Tabelle. Das nennt man kartesisches Produkt oder Cross Product.

Konzept:

```
Customers (3 Zeilen)   Orders (5 Zeilen)
```

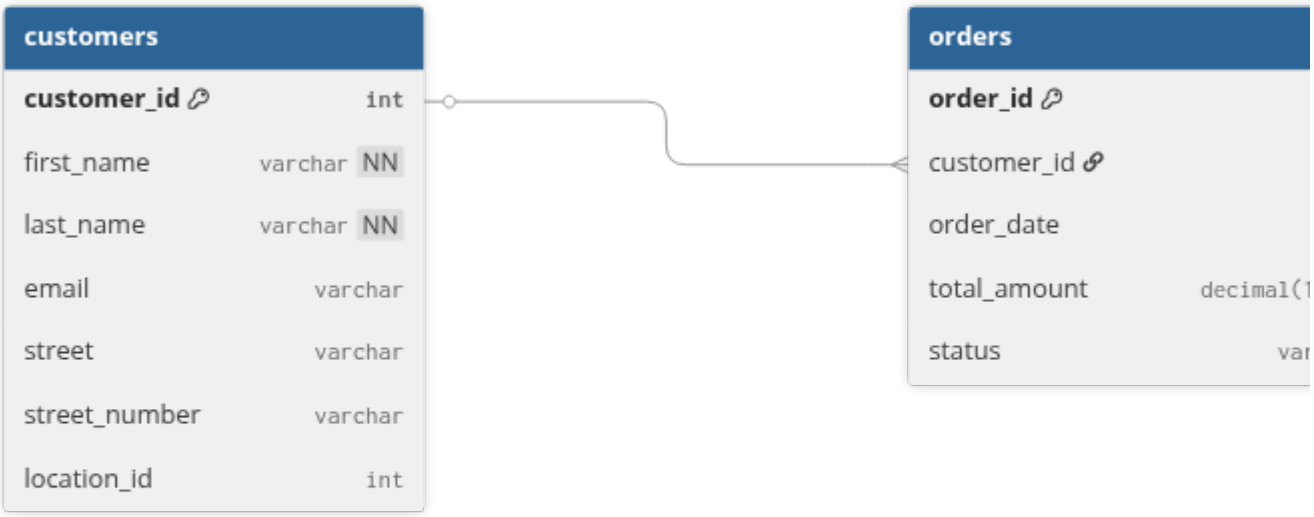
ID	Name
1	Alice
2	Bob
5	Emma

OID	CID
101	1
102	1
103	2
104	3
105	4

```
FROM customers, orders → 3 × 5 = 15 Kombinationen!
```

Jeder Kunde wird mit jeder Bestellung kombiniert – auch wenn die Bestellung gar nicht zu diesem Kunden gehört! Das ist meist nicht das, was wir wollen.

Live-Beispiel: Kartesisches Produkt im Online-Shop



dbdiagram.io

Experiment: Listen Sie Customers und Orders im FROM auf, ohne Bedingung.

```
-- VORSICHT: Kartesisches Produkt!
SELECT
  c.customer_id,
  c.first_name,
  o.order_id,
  o.customer_id AS order_customer_id
FROM customers c, orders o
LIMIT 10;  -- Nur erste 10 Zeilen zeigen
```

```
-- VORSICHT: Kartesisches Produkt!
SELECT
  c.customer_id,
  c.first_name,
  o.order_id,
  o.customer_id AS order_customer_id
FROM customers c, orders o
LIMIT 10
```

#	customer_id	first_name	order_id	order_customer_id
1	1	Alice	101	1
2	1	Alice	102	1
3	1	Alice	103	2
4	1	Alice	104	3
5	1	Alice	105	4
6	2	Bob	101	1
7	2	Bob	102	1
8	2	Bob	103	2
9	2	Bob	104	3
10	2	Bob	105	4

10 rows

```
-- Nur erste 10 Zeilen zeigen
```

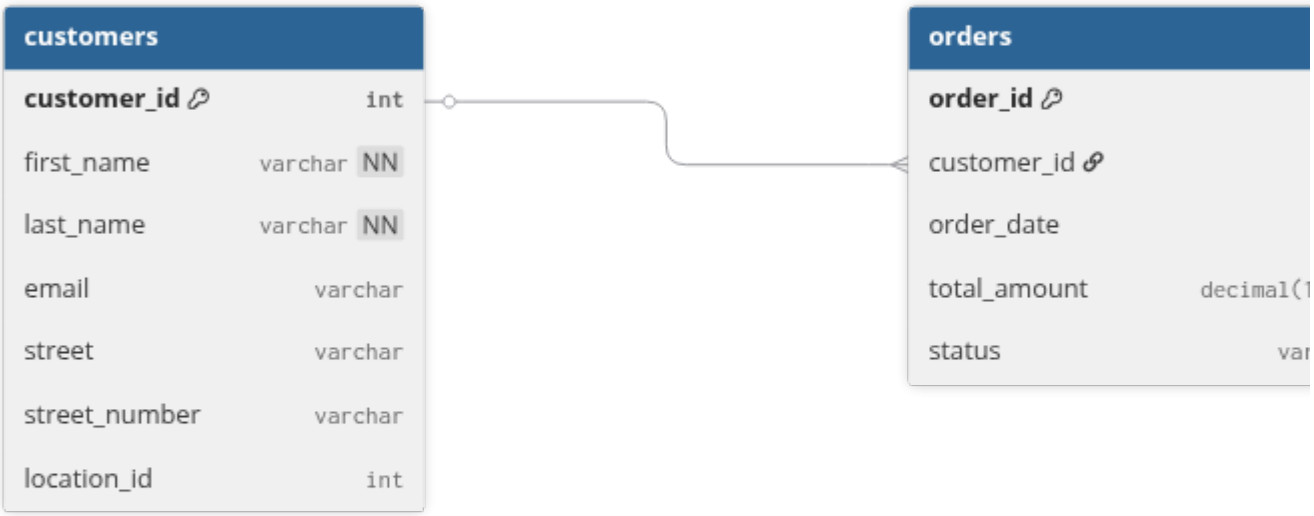
ok

Führen Sie die Query aus. Was sehen Sie? Alice (customer_id = 1) erscheint mit allen Bestellungen – auch mit Bestellungen von Bob und Carol! Das ist das kartesische Produkt: 5 Kunden × 5 Bestellungen = 25 Zeilen.

Problem: Die meisten dieser Kombinationen sind unsinnig! Alice sollte nur mit ihren eigenen Bestellungen verknüpft werden.

Die Lösung: WHERE-Bedingung

Um nur sinnvolle Kombinationen zu bekommen, filtern wir im WHERE: Verbinde nur Zeilen, wo customer_id übereinstimmt.



dbdiagram.io

```sql
-- TODO: Filtern Sie das kartesische Produkt:
-- Zeigen Sie nur Kunden mit ihren eigenen Bestellungen.
-- Tipp: c.customer_id = o.customer_id
SELECT
  customer_id,
  first_name,
  order_id,
  order_date,
  total_amount
FROM ???
```

```
-- TODO: Filtern Sie das kartesische Produkt:
-- Zeigen Sie nur Kunden mit ihren eigenen Bestellungen.
-- Tipp: c.customer_id = o.customer_id
SELECT
  customer_id,
  first_name,
  order_id,
  order_date,
  total_amount
FROM ???
```

```
syntax error at or near "???"
```

So funktioniert das:

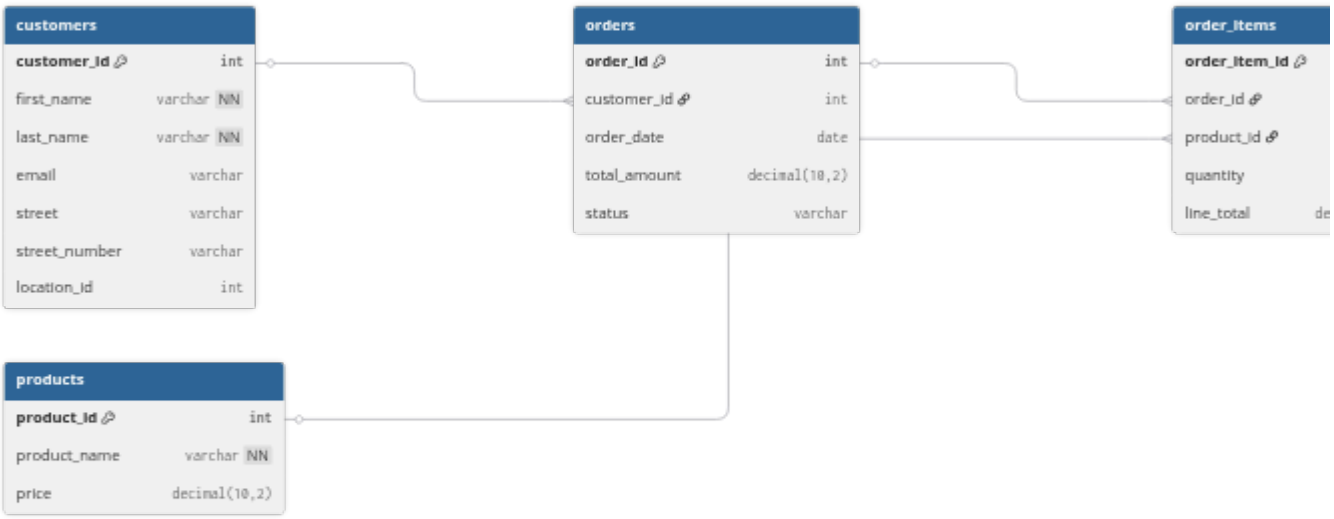
1. SQL erzeugt zunächst das kartesische Produkt (5 × 5 = 25 Zeilen)
2. Dann filtert WHERE: Nur Zeilen, wo customer_id übereinstimmt
3. Ergebnis: Nur 5 Zeilen (Kunde mit seiner Bestellung)

Das ist die klassische Methode, Tabellen zu verbinden – und genau so wurde SQL in den 1980ern geschrieben! Aber diese Syntax hat Nachteile.

Mehrere Tabellen kombinieren

Sie können beliebig viele Tabellen auflisten – aber die WHERE-Bedingungen werden schnell komplex.

Aufgabe: Zeigen Sie Kunde, Bestellung UND Produkt zusammen.



dbdiagram.io

dbdiagram.io

```sql
-- TODO: Verbinden Sie customers, orders, order_items, products
-- Hinweis: Sie brauchen 3 WHERE-Bedingungen!
SELECT
  first_name || ' ' || last_name AS customer,
  order_id,
  product_name,
  quantity
FROM ???
WHERE ???
```

```
-- TODO: Verbinden Sie customers, orders, order_items, products
-- Hinweis: Sie brauchen 3 WHERE-Bedingungen!
SELECT
  first_name || ' ' || last_name AS customer,
  order_id,
  product_name,
  quantity
FROM ???
WHERE ???
```

```
syntax error at or near "???"
```

Das funktioniert, aber:

- Die WHERE-Bedingungen mischen Join-Logik mit Filter-Logik
- Bei 4 Tabellen sind das schon 3 Bedingungen – bei 10 Tabellen?
- Vergessen Sie eine Bedingung → versehentliches kartesisches Produkt!
- Unleserlich: Was ist Join, was ist Filter?

Das Problem mit der impliziten Syntax

Diese Methode wird **implizite Join-Syntax** genannt. Sie hat mehrere Nachteile:

Problem	Beschreibung	Beispiel
Unleserlich	Join-Bedingungen vermischt mit Filter-Bedingungen	`WHERE a.id = b.id AND b.status = 'active'`
Fehleranfällig	Vergessene Bedingung → kartesisches Produkt	`FROM t1, t2, t3 WHERE t1.id = t2.id` (t3 fehlt!)
Kein LEFT/RIGHT	Outer Joins nicht möglich (ohne proprietary Syntax)	Oracle: (`+`) Notation
Veraltet	SQL-92 Standard hat explizite JOIN-Syntax eingeführt	Vor 30+ Jahren!

Deshalb gilt heute: **Nutzen Sie immer die explizite JOIN-Syntax!** Die ist moderner, klarer und mächtiger.

Warum Sie das trotzdem kennen müssen

Warum habe ich Ihnen dann die implizite Syntax gezeigt? Drei Gründe:

1. Legacy-Code verstehen

Viele alte Datenbanken und Anwendungen nutzen diese Syntax noch. Wenn Sie bestehenden Code warten, werden Sie ihr begegnen.

2. Kartesisches Produkt verstehen

Die explizite JOIN-Syntax versteckt, was wirklich passiert. Mit FROM + WHERE sehen Sie: SQL erzeugt zunächst alle Kombinationen, dann filtert es. Das hilft beim Performance-Verständnis.

3. CROSS JOIN erkennen

Wenn Sie versehentlich mehrere Tabellen auflisten ohne JOIN-Bedingung, passiert ein CROSS JOIN (kartesisches Produkt). Sie müssen das erkennen können!

```sql
-- ❌ Versehentlicher CROSS JOIN (häufiger Fehler!):
SELECT * FROM customers, orders;
-- 5 × 5 = 25 Zeilen, meist ungewollt!

-- ✅ Expliziter CROSS JOIN (wenn gewollt):
SELECT * FROM customers CROSS JOIN orders;
```

```sql
-- ❌ Versehentlicher CROSS JOIN (häufiger Fehler!):
SELECT * FROM customers, orders
```

#	customer_id	first_name	last_name	email	street
1	1	Alice	Anderson	alice@email.com	Unter den Linden
2	1	Alice	Anderson	alice@email.com	Unter den Linden
3	2	Alice	Anderson	alice@email.com	Unter den Linden
4	3	Alice	Anderson	alice@email.com	Unter den Linden
5	4	Alice	Anderson	alice@email.com	Unter den Linden
6	1	Bob	Brown	bob@email.com	Reeperbahn
7	1	Bob	Brown	bob@email.com	Reeperbahn
8	2	Bob	Brown	bob@email.com	Reeperbahn
9	3	Bob	Brown	bob@email.com	Reeperbahn
10	4	Bob	Brown	bob@email.com	Reeperbahn
11	1	Carol	Clark	carol@email.com	Marienplatz
12	1	Carol	Clark	carol@email.com	Marienplatz
13	2	Carol	Clark	carol@email.com	Marienplatz
14	3	Carol	Clark	carol@email.com	Marienplatz
15	4	Carol	Clark	carol@email.com	Marienplatz
16	1	David	Davis	david@email.com	Hohe Straße
17	1	David	Davis	david@email.com	Hohe Straße
18	2	David	Davis	david@email.com	Hohe Straße
19	3	David	Davis	david@email.com	Hohe Straße
20	4	David	Davis	david@email.com	Hohe Straße
21	1	Emma	Evans	emma@email.com	Zeil
22	1	Emma	Evans	emma@email.com	Zeil
23	2	Emma	Evans	emma@email.com	Zeil
24	3	Emma	Evans	emma@email.com	Zeil

25	4	Emma	Evans	emma@email.com	Zeil

25 rows

```
-- 5 × 5 = 25 Zeilen, meist ungewollt!

-- ✅ Expliziter CROSS JOIN (wenn gewollt):
SELECT * FROM customers CROSS JOIN orders
```

#	customer_id	first_name	last_name	email	street
1	1	Alice	Anderson	alice@email.com	Unter den Linden
2	1	Alice	Anderson	alice@email.com	Unter den Linden
3	2	Alice	Anderson	alice@email.com	Unter den Linden
4	3	Alice	Anderson	alice@email.com	Unter den Linden
5	4	Alice	Anderson	alice@email.com	Unter den Linden
6	1	Bob	Brown	bob@email.com	Reeperbahn
7	1	Bob	Brown	bob@email.com	Reeperbahn
8	2	Bob	Brown	bob@email.com	Reeperbahn
9	3	Bob	Brown	bob@email.com	Reeperbahn
10	4	Bob	Brown	bob@email.com	Reeperbahn
11	1	Carol	Clark	carol@email.com	Marienplatz
12	1	Carol	Clark	carol@email.com	Marienplatz
13	2	Carol	Clark	carol@email.com	Marienplatz
14	3	Carol	Clark	carol@email.com	Marienplatz
15	4	Carol	Clark	carol@email.com	Marienplatz
16	1	David	Davis	david@email.com	Hohe Straße
17	1	David	Davis	david@email.com	Hohe Straße
18	2	David	Davis	david@email.com	Hohe Straße
19	3	David	Davis	david@email.com	Hohe Straße
20	4	David	Davis	david@email.com	Hohe Straße
21	1	Emma	Evans	emma@email.com	Zeil
22	1	Emma	Evans	emma@email.com	Zeil
23	2	Emma	Evans	emma@email.com	Zeil
24	3	Emma	Evans	emma@email.com	Zeil

25	4	Emma	Evans	emma@email.com	Zeil

25 rows

Vergleich: Implizit vs. Explizit

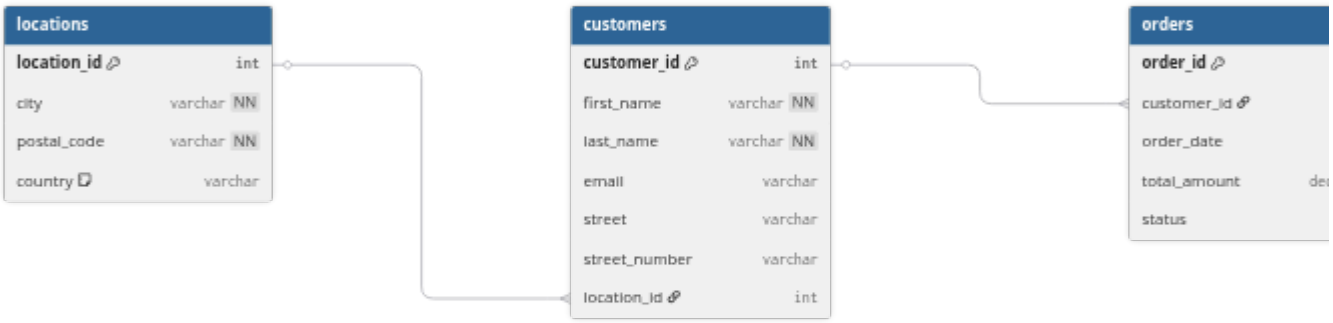
Schauen wir uns beide Syntaxen direkt nebeneinander an – für die gleiche Aufgabe.

Implizite Syntax (veraltet)	Explizite Syntax (modern)
`FROM customers c, orders o`	`FROM customers c INNER JOIN orders o`
`WHERE c.customer_id = o.customer_id`	`ON c.customer_id = o.customer_id`
Filter UND Join vermischt	Join getrennt von Filter
Kein LEFT/RIGHT JOIN möglich	Alle Join-Typen verfügbar
Kartesisches Produkt bei Fehler	Fehler bei fehlender ON-Bedingung

Faustregel: Implizite Syntax = INNER JOIN ohne `ON`. Mehr geht nicht.

Übung: Implizit

Aufgabe: Zeigen Sie Vorname, Stadt und Bestelldatum für alle abgeschlossenen Bestellungen.



dbdiagram.io

dbdiagram.io

```sql
-- Gegeben (implizit):
SELECT
  first_name,
  city,
  order_date
FROM ???
WHERE ???
```

```
-- Gegeben (implizit):
SELECT
  first_name,
  city,
  order_date
FROM ???
WHERE ???
```

```
syntax error at or near "???"
```

Zusammenfassung: FROM mit mehreren Tabellen

Was passiert intern:

1. SQL erzeugt das kartesische Produkt aller Tabellen im FROM
2. WHERE filtert dann die gewünschten Kombinationen
3. Das ist ineffizient – aber so funktioniert die logische Verarbeitung

Warum explizite JOINs besser sind:

- ✅ Klar getrennt: Join-Bedingungen (`ON`) vs. Filter (`WHERE`)
- ✅ Alle Join-Typen verfügbar (`LEFT`, `RIGHT`, `FULL OUTER`)
- ✅ Weniger fehleranfällig (kein versehentliches kartesisches Produkt)
- ✅ Bessere Performance-Optimierung durch Query Planner

> **Best Practice:** Nutzen Sie immer `JOIN ... ON` statt `FROM ..., ... WHERE`!

Jetzt, wo Sie verstehen, was im Hintergrund passiert, schauen wir uns die ersten echten Abfrage-Techniken an: Subqueries!

Technik 1: Subqueries (Verschachtelte SELECT)

Der erste Ansatz, um Daten aus verschiedenen Tabellen zu kombinieren, sind Subqueries – verschachtelte SELECT-Statements. Das fühlt sich natürlich an: „Ich brauche Daten aus Tabelle B, um Tabelle A zu filtern."

Aber was ist eine Subquery genau? Und wo können wir sie überall einsetzen?

Was ist eine Subquery?

Eine Subquery ist ein SELECT-Statement, das innerhalb eines anderen SQL-Statements ausgeführt wird. Statt erst eine Query auszuführen, das Ergebnis zu notieren und dann in einer zweiten Query zu verwenden, verschachteln wir beide.

Konzept:

```sql
-- Ohne Subquery (zwei Schritte):
-- Schritt 1: Welche customer_ids haben Bestellungen?
SELECT DISTINCT customer_id FROM orders;
-- Ergebnis: 1, 2, 3, 4

-- Schritt 2: Zeige diese Kunden
SELECT * FROM customers WHERE customer_id IN (1, 2, 3, 4);
```

```
-- Ohne Subquery (zwei Schritte):
-- Schritt 1: Welche customer_ids haben Bestellungen?
SELECT DISTINCT customer_id FROM orders
```

#	customer_id
1	1
2	2
3	3
4	4

4 rows

```
-- Ergebnis: 1, 2, 3, 4

-- Schritt 2: Zeige diese Kunden
SELECT * FROM customers WHERE customer_id IN (1, 2, 3, 4)
```

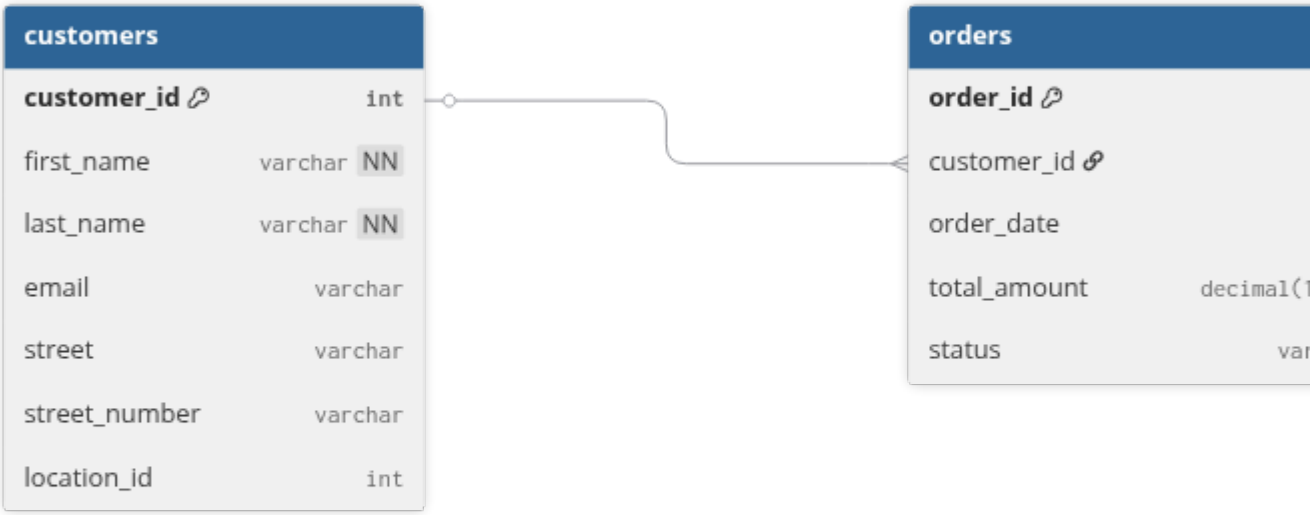
#	customer_id	first_name	last_name	email	street
1	1	Alice	Anderson	alice@email.com	Unter den Linden
2	2	Bob	Brown	bob@email.com	Reeperbahn
3	3	Carol	Clark	carol@email.com	Marienplatz
4	4	David	Davis	david@email.com	Hohe Straße

4 rows

Mit Subquery (ein Schritt):

```sql
SELECT * FROM "TABLE"
WHERE id IN (
  SELECT ... FROM "OTHER TABLE"
```

```
);
```



dbdiagram.io

dbdiagram.io

```sql
-- TODO: Schreiben sie die Bestellung um und ermitteln sie alle Kunden eine Bestellung haben.
SELECT *
FROM customers c, orders o
WHERE c.customer_id = o.customer_id;
```

```
-- TODO: Schreiben sie die Bestellung um und ermitteln sie alle Kunden,
die eine Bestellung haben.
SELECT *
FROM customers c, orders o
WHERE c.customer_id = o.customer_id
```

#	customer_id	first_name	last_name	email	street
1	1	Alice	Anderson	alice@email.com	Unter den Linden
2	1	Alice	Anderson	alice@email.com	Unter den Linden
3	2	Bob	Brown	bob@email.com	Reeperbahn
4	3	Carol	Clark	carol@email.com	Marienplatz
5	4	David	Davis	david@email.com	Hohe Straße

5 rows

Die innere Query (Subquery) wird zuerst ausgeführt. Ihr Ergebnis wird dann von der äußeren Query verwendet.

Subquery-Typen: Übersicht

Je nachdem, was eine Subquery zurückgibt, unterscheiden wir verschiedene Typen:

Subquery-Typ	Rückgabewert	Beispiel-Operator	Use Case
Scalar Subquery	Ein einzelner Wert	`=`, `>`, `<`	Durchschnitt, Maximum vergleichen
Row Subquery	Eine Zeile (mehrere Spalten)	`= (col1, col2)`	Selten, Multi-Column-Vergleich
Table Subquery	Mehrere Zeilen, eine Spalte	`IN`, `ANY`, `ALL`	Filtern mit Liste
Derived Table	Mehrere Zeilen/Spalten	Im `FROM`	Komplexe Aggregationen
Correlated	Referenziert äußere Query	Mit Spalte aus äußerer Q.	Pro-Zeile-Berechnung

Schauen wir uns jetzt die wichtigsten dieser Typen im Detail an, beginnend mit dem häufigsten: WHERE Subqueries.

WHERE Subqueries: Filtern mit Ergebnissen aus anderen Tabellen

Die häufigste Form: Eine Subquery im WHERE liefert Werte zum Filtern.

Aufgabe: Zeigen Sie alle Kunden, die mindestens eine Bestellung haben.

```sql
-- Ohne Subquery (zwei Schritte):
-- Schritt 1: Welche customer_ids haben Bestellungen?
SELECT DISTINCT customer_id FROM orders;
-- Ergebnis: 1, 2, 3, 4

-- Schritt 2: Zeige diese Kunden
SELECT * FROM customers WHERE customer_id IN (1, 2, 3, 4);
```

```
-- Ohne Subquery (zwei Schritte):
-- Schritt 1: Welche customer_ids haben Bestellungen?
SELECT DISTINCT customer_id FROM orders
```

#	customer_id
1	1
2	2
3	3
4	4

4 rows

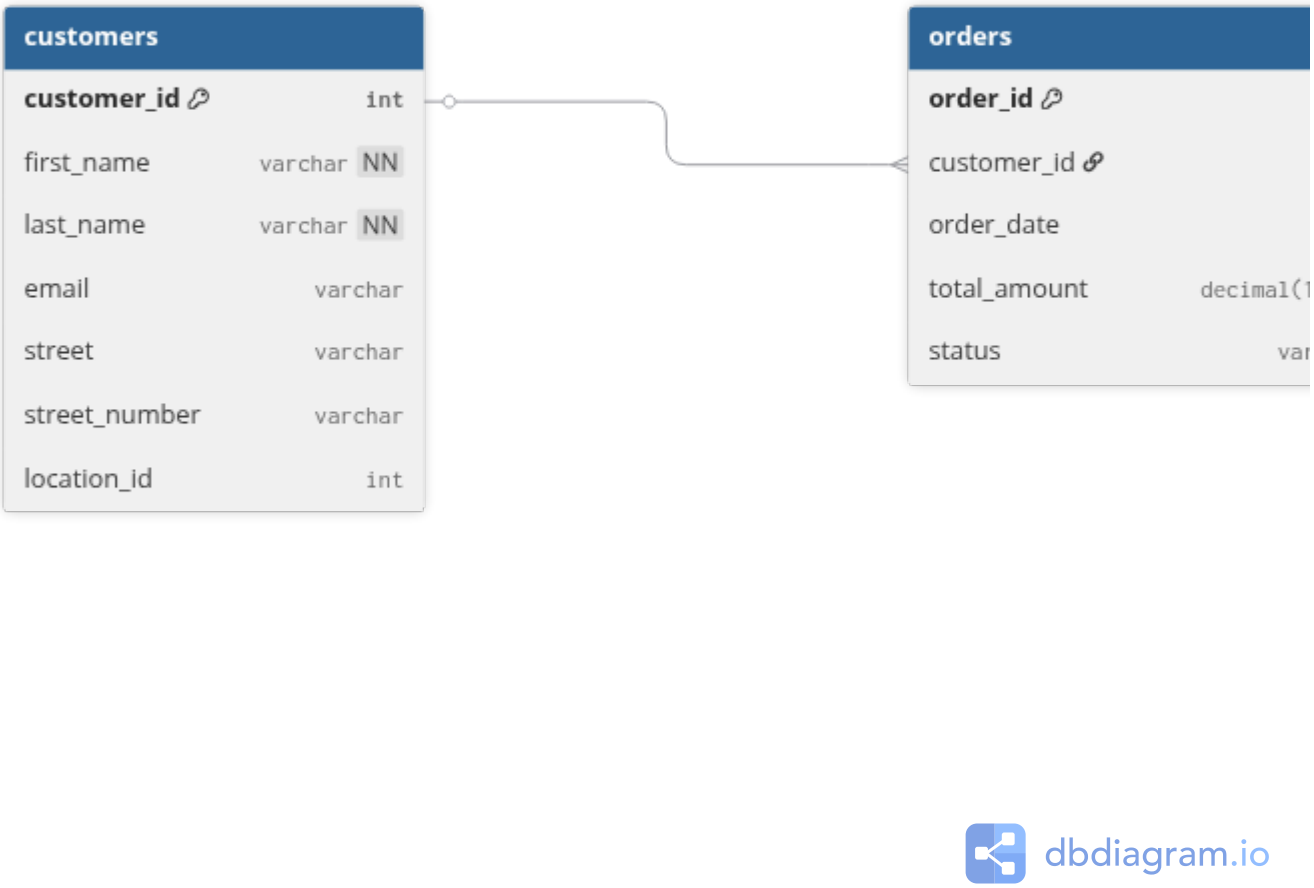
```
-- Ergebnis: 1, 2, 3, 4

-- Schritt 2: Zeige diese Kunden
SELECT * FROM customers WHERE customer_id IN (1, 2, 3, 4)
```

#	customer_id	first_name	last_name	email	street
1	1	Alice	Anderson	alice@email.com	Unter den Linden
2	2	Bob	Brown	bob@email.com	Reeperbahn
3	3	Carol	Clark	carol@email.com	Marienplatz
4	4	David	Davis	david@email.com	Hohe Straße

4 rows

```sql
SELECT
  column_1,
  column_2,
  ...
FROM table_1
WHERE column_x IN (
  -- Subquery
  SELECT column
  FROM table_n
  WHERE condition
);
```



dbdiagram.io

```
-- TODO: Verändern sie die folgende Query, sodass sie eine Subquery im
  nutzt,
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  c.email
FROM customers c, orders o
WHERE c.customer_id = o.customer_id;
```

```
-- TODO: Verändern sie die folgende Query, sodass sie eine Subquery im
WHERE nutzt,
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  c.email
FROM customers c, orders o
WHERE c.customer_id = o.customer_id
```

#	customer_id	first_name	last_name	email
1	1	Alice	Anderson	alice@email.com
2	1	Alice	Anderson	alice@email.com
3	2	Bob	Brown	bob@email.com
4	3	Carol	Clark	carol@email.com
5	4	David	Davis	david@email.com

5 rows

Wie funktioniert das?

1. Die **innere Query** (Subquery) wird zuerst ausgeführt: `SELECT customer_id FROM orders`
2. Ergebnis: Liste von customer_ids, die Bestellungen haben: `(1, 1, 2, 3, 4)`
3. Die **äußere Query** filtert damit: `WHERE customer_id IN (1, 1, 2, 3, 4)`

Das ist einfach zu lesen und zu verstehen. Aber Emma (customer_id = 5) fehlt – die hat keine Bestellung. Subqueries im WHERE sind gut für „zeige mir nur die mit…“

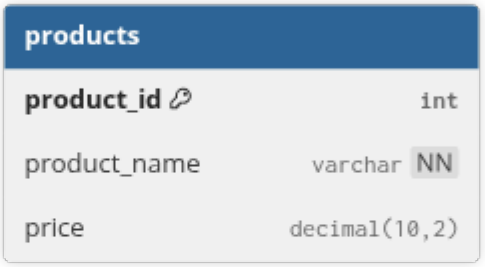
Scalar Subqueries: Einzelwerte berechnen

Eine Subquery kann auch einen einzelnen Wert zurückgeben – zum Vergleichen oder Berechnen.

```
SELECT
    column_a,
    column_b,
    ...,
    (
        -- Subquery: liefert einen Wert (z. B. Durchschnitt)
        SELECT AGG(target_column)
        FROM source_table
    ) AS computed_value
FROM main_table
```

```
WHERE filter_column > (
        -- Subquery: derselbe Wert für die WHERE-Bedingung
        SELECT AGG(target_column)
        FROM source_table
    );
```

Aufgabe: Zeigen Sie alle Produkte, die teurer sind als der Durchschnittspreis.



dbdiagram.io

```
1  SELECT
2    product_id,
3    product_name,
4    price,
5    ??? AS avg_price
6  FROM products
7  WHERE ???;
```

```
SELECT
  product_id,
  product_name,
  price,
  ??? AS avg_price
FROM products
WHERE ???
```

```
syntax error at or near "AS"
```

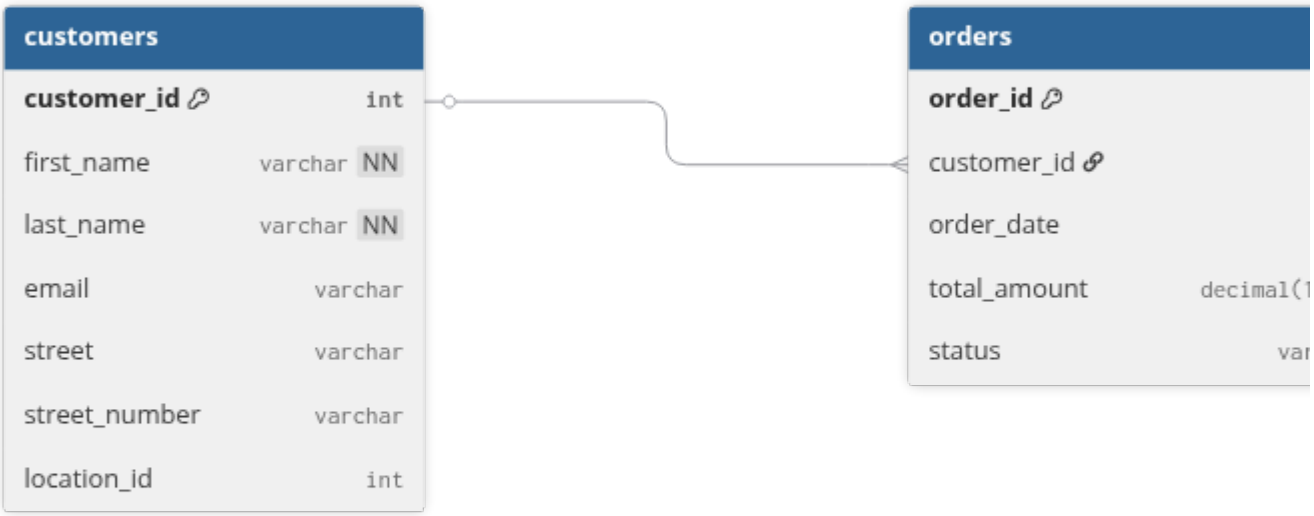
Problem: Wir berechnen den Durchschnitt zweimal! Subquery in SELECT UND in WHERE. Das ist ineffizient und schwer wartbar.
Scalar Subqueries sind nützlich, aber wenn Sie denselben Wert mehrfach brauchen, wird es unübersichtlich. Später sehen wir, wie CTEs dieses Problem lösen.

Subqueries in SELECT: Spalten aus anderen Tabellen

Sie können Subqueries auch nutzen, um zusätzliche Spalten zu berechnen.

```
SELECT
    column_1,
    column_2,
    ...,
    (
        -- Subquery: berechnet einen Wert pro Zeile der äußeren Tabelle
        SELECT AGG(*)
        FROM inner_table
        WHERE inner_table.foreign_key = outer_table.primary_key
    ) AS computed_value
FROM outer_table;
```

Aufgabe: Zeigen Sie für jeden Kunden die Anzahl seiner Bestellungen.



dbdiagram.io

dbdiagram.io

```
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  (???)
FROM customers c;
```

```
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  (???)
FROM customers c
```

```
syntax error at or near ")"
```

Das ist eine correlated Subquery – sie referenziert die äußere Query (`c.customer_id`). Für jeden Kunden wird die Subquery neu ausgeführt.

Funktioniert, aber: Bei 10.000 Kunden wird die Subquery 10.000 Mal ausgeführt! Performance-Problem.

Subquery-Grenzen: Wann wird es problematisch?

Subqueries sind intuitiv, aber sie haben Grenzen:

Problem	Beschreibung	Beispiel
Unleserlich	Verschachtelte Queries sind schwer zu verstehen	3+ Ebenen Verschachtelung
Nicht wiederverwendbar	Berechnete Werte können nicht mehrfach genutzt werden	Durchschnitt 2× berechnen
Performance	Correlated Subqueries werden oft wiederholt ausgeführt	10.000 Kunden = 10.000 Subqueries
Keine parallelen Spalten	Schwierig, Spalten aus mehreren Tabellen parallel zu zeigen	Kunde + Bestellung + Produkt

Es muss einen besseren Weg geben! Und den gibt es: CTEs (Common Table Expressions).

Technik 2: CTEs (WITH) – Benannte Zwischenergebnisse

CTEs sind „benannte Subqueries“. Sie machen Queries lesbarer und wiederverwendbar. Statt alles in einer verschachtelten Monster-Query zu schreiben, teilen Sie es in logische Schritte auf.

CTE-Syntax: WITH … AS

Todo